



操作系统实验报告

实验二：进程优先级管理与信号量实现

专 业： 人工智能

姓 名： 张晓宇

学 号： 37220232203927

实验日期： 2025.11.7

指导教师： 曹冬林

2025 年 12 月 6 日

目录

1 实验目的

本次实验围绕操作系统的两个核心主题——进程管理（侧重于优先级）和进程同步（侧重于信号量）展开，旨在达成以下目标：

1.1 深入理解进程管理与调度

学习如何在操作系统内核中实现进程优先级的查看和修改机制。理解优先级如何作为进程调度的重要依据，为从默认的轮转调度（Round Robin）转向优先级调度奠定基础。

1.2 掌握内核数据结构的扩展与系统调用

学习如何修改关键的内核数据结构，特别是进程控制块（`struct proc`），以包含进程优先级信息；学习如何定义新的内核数据结构来实现信号量（`struct semaphore`）。同时，通过实现 `getptable()`、`setpriority()`、`sem_init()` 等一系列系统调用，熟练掌握从用户态发起请求到内核态响应处理的完整流程。

1.3 深入理解进程同步与互斥

分析多进程并发访问共享资源（如文件）时可能出现的竞态条件（Race Condition）和数据不一致问题。理解临界区（Critical Section）的概念以及保护临界区的必要性。

1.4 掌握信号量机制的原理与实现

学习 Dijkstra 提出的信号量机制（Semaphore），理解 P 操作（`wait`）和 V 操作（`signal`）的原理。掌握如何在内核中利用 `sleep()` 和 `wakeup()` 原语来实现信号量的阻塞和唤醒功能。

2 实验环境

2.1 硬件环境

- CPU: Intel i9-13900h
- GPU: NVIDIA RTX 4050
- 宿主机操作系统: Windows 11

2.2 软件环境

- 操作系统: Ubuntu 18.04 (64 位)
- 模拟器: QEMU
- 编译工具: gcc, make (build-essential)
- 目标操作系统内核: xv6

3 实验要求

3.1 任务一：进程优先级管理

1. 实现 **ps** 命令 (ps.c): 显示系统进程列表, 包括 PID, PPID, PRI (优先级), MEM, STATE, CMD。
2. 实现系统调用 **gettable()**: 用于从内核获取进程表信息。
3. 实现 **nice** 命令 (nice.c): 用户态程序, 用于修改进程优先级。命令格式: **nice pid priority** (优先级范围 [1, 20], 数值越小优先级越高)。
4. 实现系统调用 **setpriority(pid, priority)**: 供 nice 命令调用。

3.2 任务二：信号量实现

在内核中实现以下系统调用:

1. `int sem_init(int sem, int value)`: 初始化信号量 `sem` 的值为 `value`。
2. `int sem_destroy(int sem)`: 销毁信号量 `sem`。
3. `int sem_wait(int sem, int count)`: P 操作, 申请 `count` 个资源, 如果不足则阻塞。
4. `int sem_signal(int sem, int count)`: V 操作, 释放 `count` 个资源, 并唤醒等待的进程。

3.3 任务三：使用信号量解决文件读写互斥

1. 创建 `sem_test.c`。
2. 启动 10 个并发进程。
3. 每个进程重复 50 次以下操作: 从文件中读取数值, 加 1, 写回文件。
4. 使用任务二实现的信号量保证操作的互斥性, 验证最终文件中的数值为 500。

4 实验流程

4.1 实验准备：内核基础修改

本次实验涉及大量的内核修改，特别是对进程管理和系统调用的扩展。首先需要进行一些基础准备工作。

4.1.1 扩展进程控制块 (struct proc)

为了支持优先级，必须在内核中存储每个进程的优先级信息。这需要修改 `proc.h` 文件中的 `struct proc` 定义。

同时，需要在 `proc.c` 的 `allocproc()` 函数中为新进程设置默认优先级（例如设为 10）。

```
107     found:
108         p->state = EMBRYO;
109         p->priority = 10;
110         p->pid = nextpid++;
111
```

图 1 修改 `proc.h` 和 `allocproc()` 设置默认优先级

4.1.2 系统调用实现流程

在 `xv6` 中添加一个新的系统调用通常需要修改以下几个文件：`syscall.h`, `syscall.c`, `usys.S`, `user.h`, 和 `sysproc.c`。

4.2 任务一：进程优先级管理

4.2.1 实现思路与流程

任务一的目标是实现进程优先级的查看 (`ps / getptable`) 和修改 (`nice / setpriority`)。任务一的总体流程如图??所示。

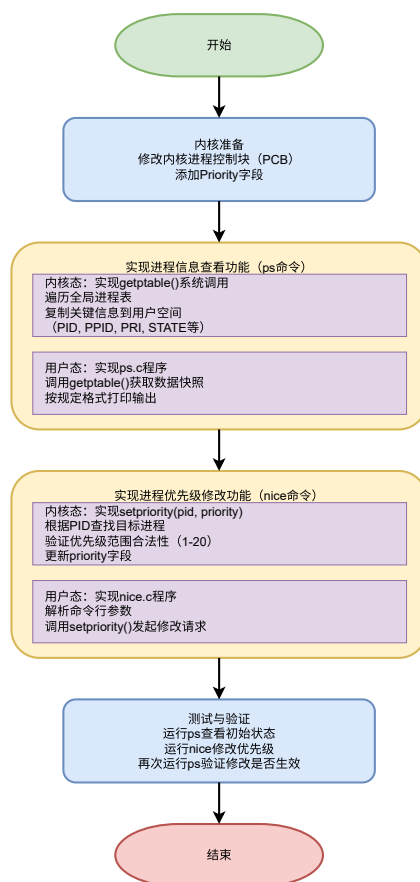


图 2 任务一总体实现流程

4.2.2 步骤一：实现 setpriority() 系统调用

该系统调用用于修改指定 PID 进程的优先级。内核实现需要完成参数获取、参数校验、获取 `ptable.lock` 锁、遍历进程表并修改优先级。

`setpriority()` 系统调用的实现流程如图??所示。

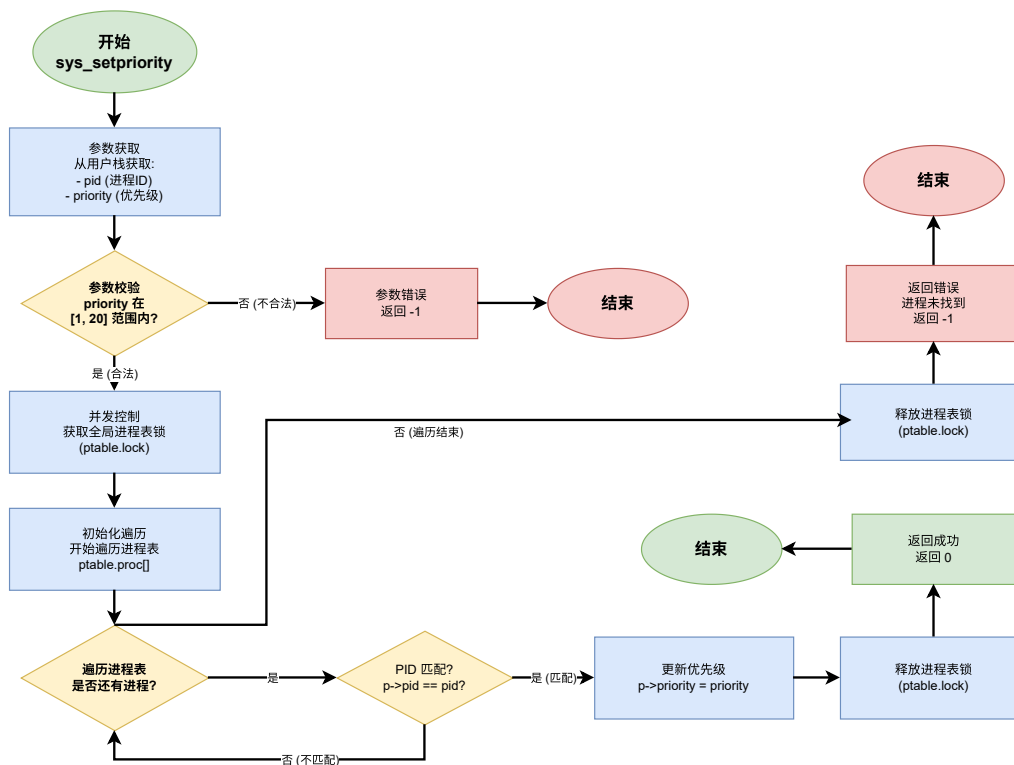


图 3 setpriority 系统调用实现流程

4.2.3 步骤二：实现 getptable() 系统调用

该系统调用用于将内核的进程表信息传递给用户态的 `ps` 命令。由于内核和用户空间的地址空间隔离，需要将内核数据复制到用户空间提供的缓冲区。

`getptable()` 系统调用的实现流程如图??所示。

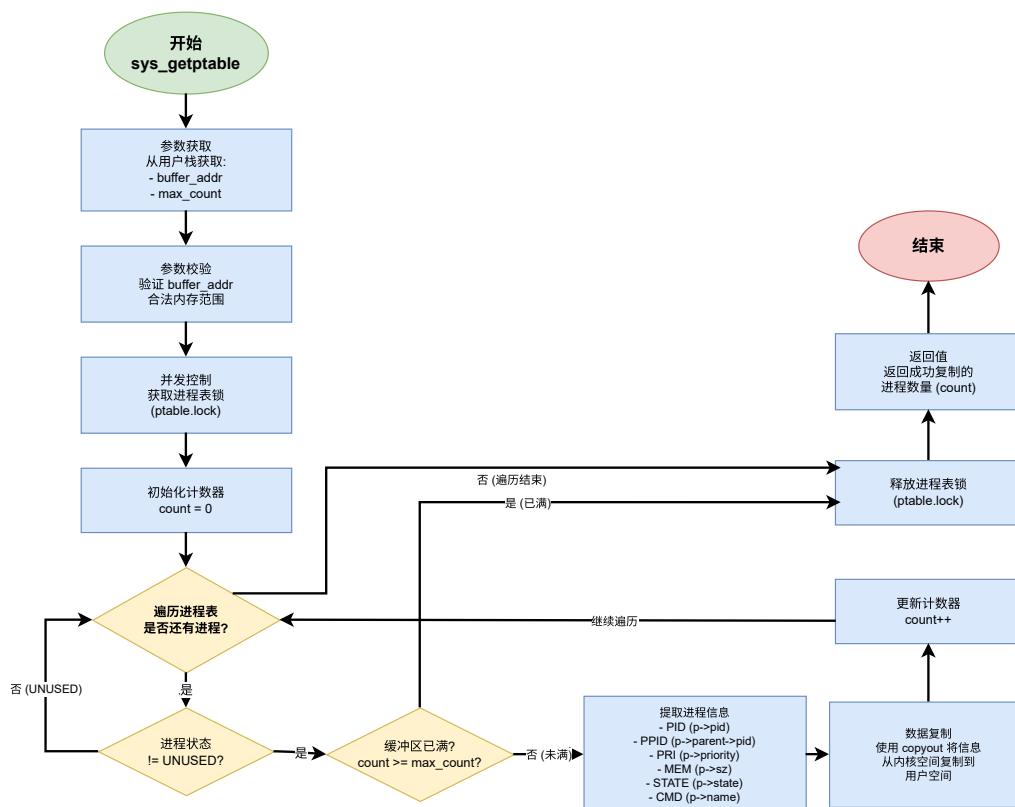


图 4 gettable 系统调用实现流程

4.2.4 步骤三：实现用户态程序 nice.c

nice 命令解析命令行参数并通过 `setpriority()` 系统调用来修改进程优先级。用户态程序 nice 的实现流程如图??所示。

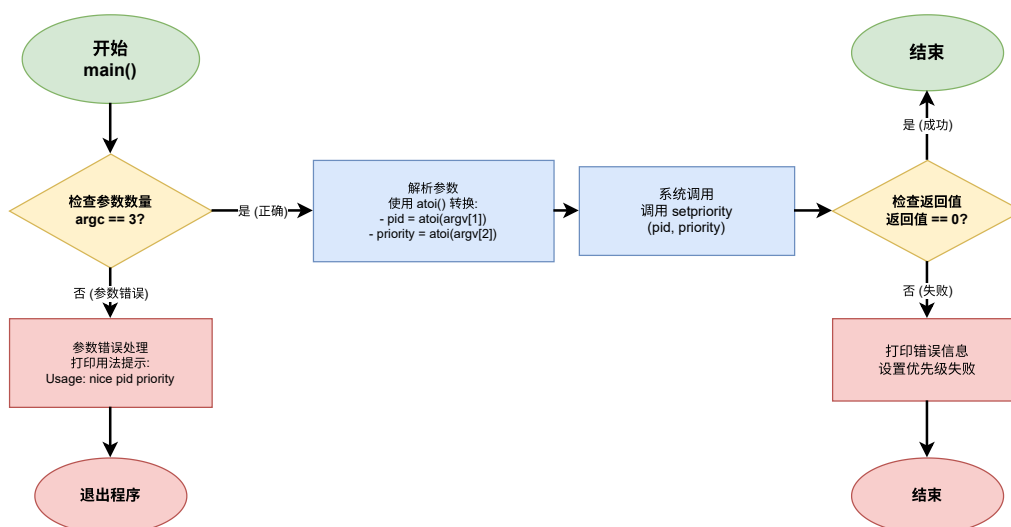


图 5 nice 用户态程序实现流程

4.2.5 步骤四：实现用户态程序 ps.c

ps 命令调用 `gettable()` 系统调用获取进程信息，并格式化输出。
用户态程序 ps 的实现流程如图??所示。

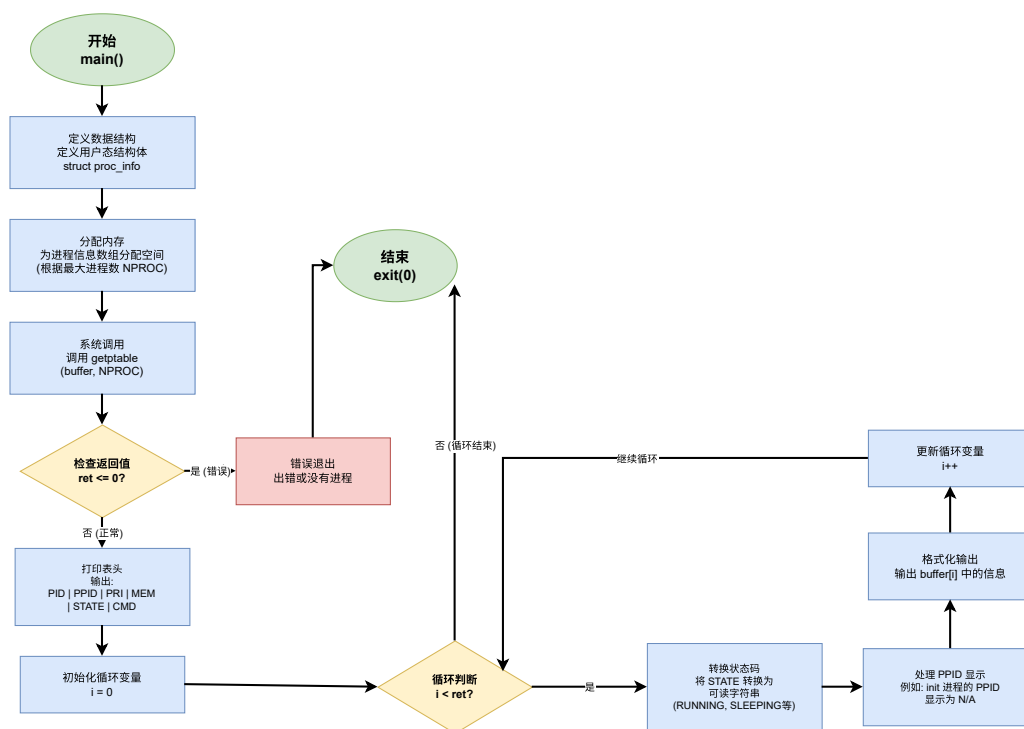


图 6 ps 用户态程序实现流程

4.2.6 测试与验证

编译并启动内核 `make qemu-nox`。

测试用例 1：验证 ps 命令

操作：ps

预期结果：输出包含 PRI 列的进程信息。init 和 sh 的默认优先级应为 10。

```

lsh> ps
PID    PPID    PRI     MEM   STATE    CMD
1      N/A     10    12288   SLEEPING  init
2       1      10    49152   SLEEPING  lsh
3       2      10    12288   RUNNING   ps
  
```

图 7 命令执行结果

测试用例 2：验证 nice 命令

操作：先用 ps 查看 PID，然后执行 `nice25`。

预期结果：再次执行 ps，PID 2 的 PRI 列应显示为 5。

```
lsh> ps
PID    PPID    PRI     MEM   STATE    CMD
1      N/A     10    12288   SLEEPING  init
2       1      10    49152   SLEEPING  lsh
3       2      10    12288   RUNNING   ps
lsh> nice 2 5
nice: priority of process 2 set to 5
lsh> ps
PID    PPID    PRI     MEM   STATE    CMD
1      N/A     10    12288   SLEEPING  init
2       1       5    49152   SLEEPING  lsh
5       2      10    12288   RUNNING   ps
```

图 8 nice 命令执行及验证结果

4.3 任务二：信号量实现

任务二要求在内核中实现信号量机制。这需要利用内核提供的同步原语 `sleep()` 和 `wakeup()`。

4.3.1 实现思路与数据结构

任务二的总体流程如图??所示。

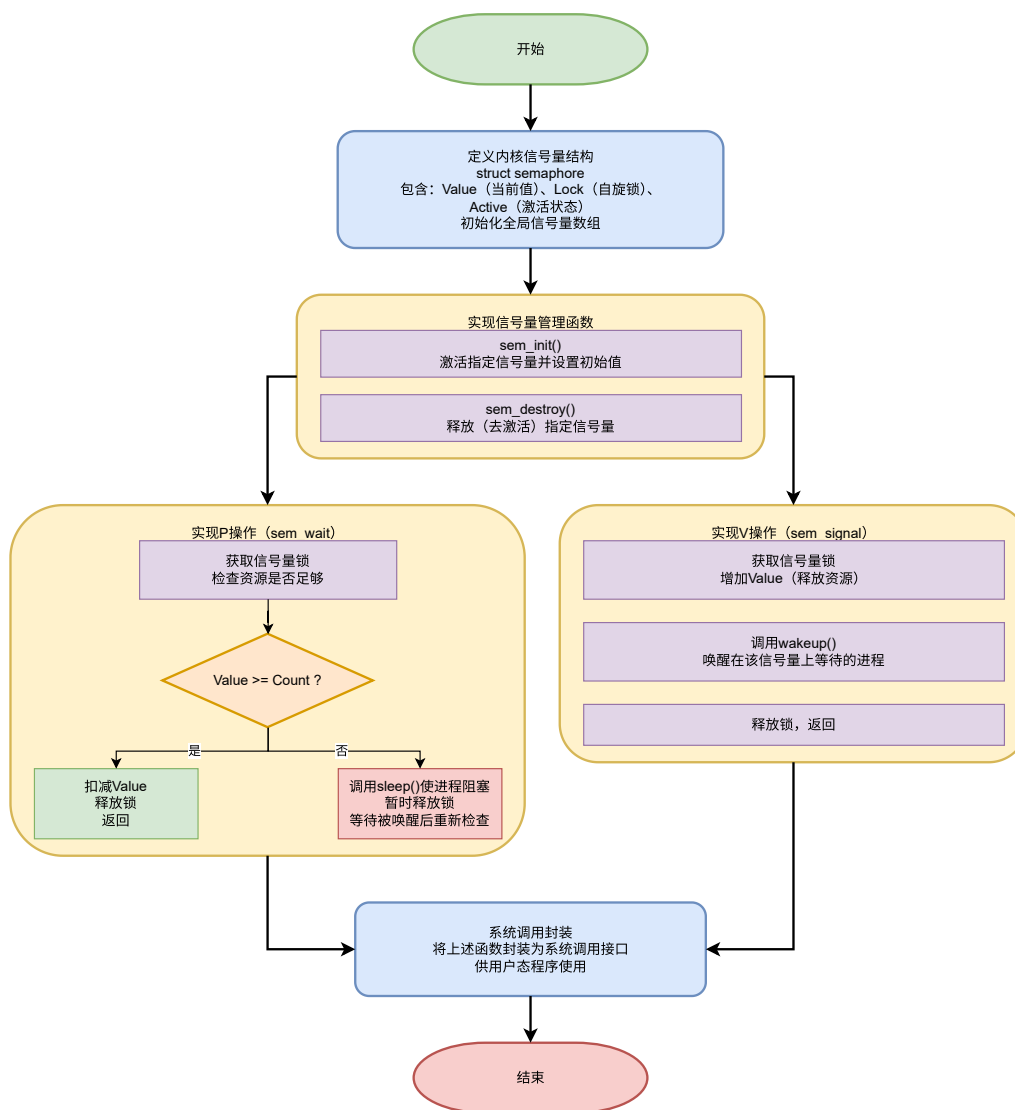


图 9 任务二总体实现流程

4.3.2 步骤一：定义信号量数据结构与初始化

在内核中定义信号量结构体，并创建全局信号量数组。

● ● ●

信号量数据结构定义

```

1      #include "spinlock.h"
2      #define MAX_SEMAPHORES 16 // 系统最大支持的信号量数量
3
4      // (定义在 proc.h 或相关头文件中)
5      struct semaphore {
  
```

```
● ● ● 信号量数据结构定义

6         int value;                // 信号量的计数值
7         struct spinlock lock;    // 保护信号量的自旋锁
8         int active;              // 标记该信号量是否正在使用
9     };

10
11     // (在 proc.c 中定义全局数组)
12     struct semaphore semaphores[MAX_SEMAPHORES];
```

在系统启动时（例如在 `main.c` 中调用 `seminit()`），需要初始化这些信号量的锁。

4.3.3 步骤二：实现 `sem_init()` 和 `sem_destroy()`

这两个系统调用用于管理信号量的生命周期。

```
● ● ● sysproc.c (sys_sem_init)

1         int
2         sys_sem_init(void)
3     {
4         int sem, value;
5         // ... (参数获取与校验) ...
6
7         // 初始化操作
8         acquire(&semaphores[sem].lock);
9         semaphores[sem].value = value;
10        semaphores[sem].active = 1;
11        release(&semaphores[sem].lock);
12
13        return 0;
14    }
```

`sem_init()` 的实现流程如图??所示。

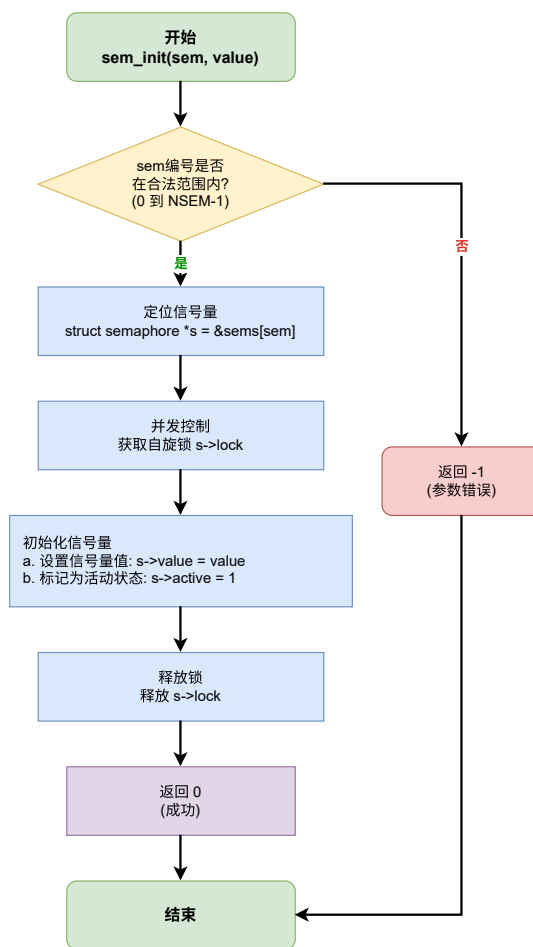


图 10 sem_init 实现流程

`sem_destroy()` 的实现流程如图??所示。

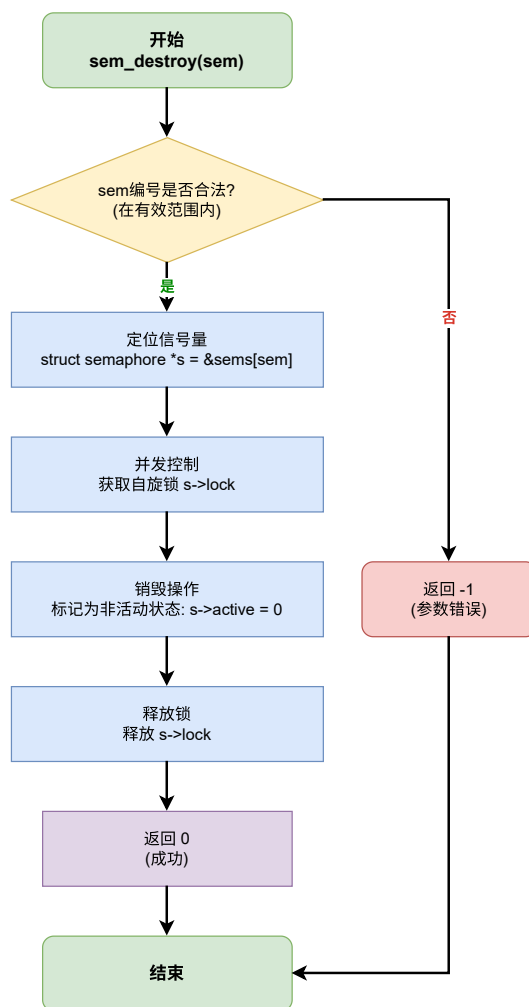


图 11 sem_destroy 实现流程

4.3.4 步骤三：实现 sem_wait() (P 操作)

sem_wait 是核心操作。如果资源不足，进程必须阻塞。关键在于使用 while 循环检查条件，并调用 sleep(chan, lock) 进行阻塞。

```

sysproc.c (sys_sem_wait)

1      int
2      sys_sem_wait(void)
3      {
4          int sem, count;
5          // ... (参数获取与校验) ...

```

```
sysproc.c (sys_sem_wait)

6
7     acquire(&semaphores[sem].lock);
8
9     // 当资源不足时，阻塞等待（使用 while 而非 if 是关键
10    )
11    while(semaphores[sem].value < count){
12        // 使用信号量的地址作为等待通道 (chan)
13        // sleep 会原子性地释放 lock 并使当前进程进入
14        SLEEPING 状态
15        sleep(&semaphores[sem], &semaphores[sem].lock);
16        // 被唤醒后，sleep 会重新获取 lock
17    }
18
19    // 资源足够，分配资源
20    semaphores[sem].value -= count;
21
22    release(&semaphores[sem].lock);
23
24    return 0;
25 }
```

`sem_wait()` (P 操作) 的实现流程如图??所示。

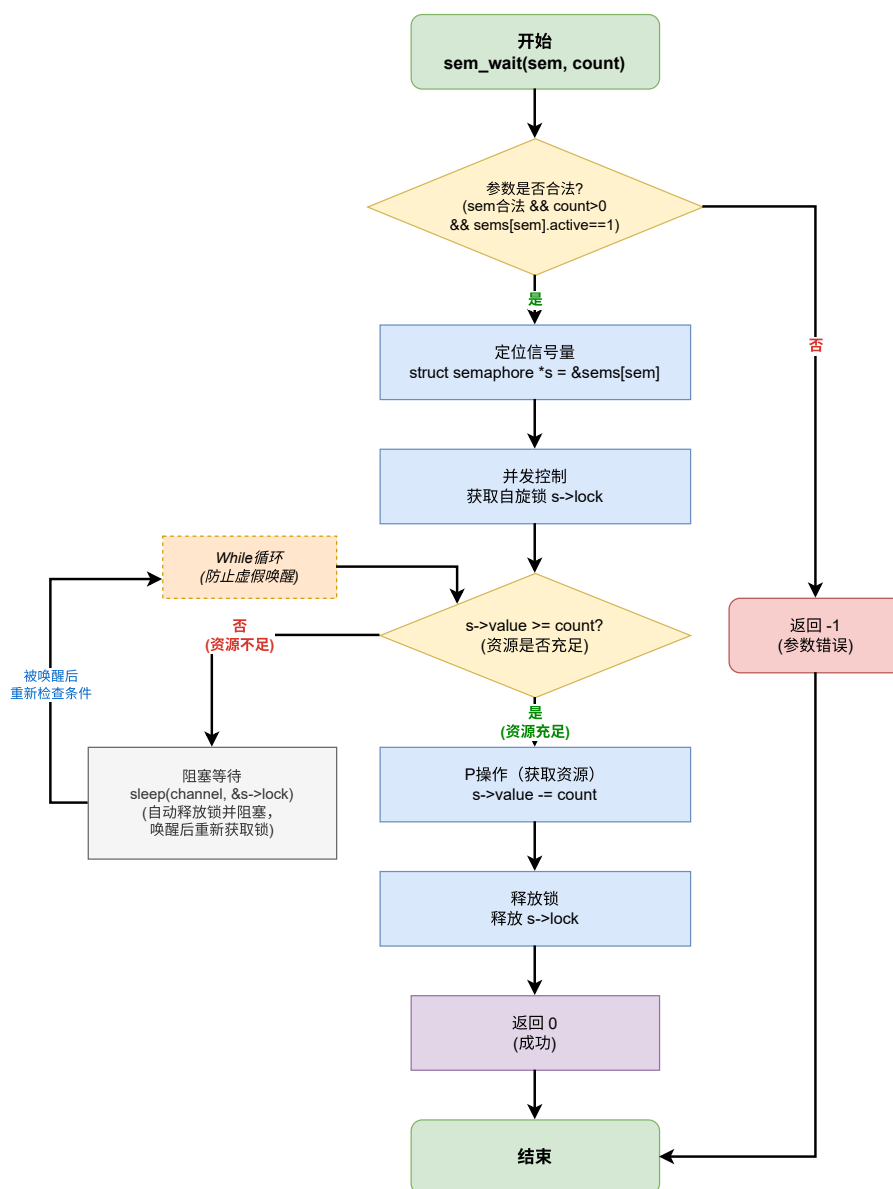


图 12 sem_wait (P 操作) 实现流程

4.3.5 步骤四：实现 sem_signal() (V 操作)

sem_signal 用于释放资源，并唤醒可能正在等待该资源的进程。

```

sysproc.c (sys_sem_signal)

1  int
2  sys_sem_signal(void)
3  {

```



```
sysproc.c (sys_sem_signal)

4      int sem, count;
5      // ... (参数获取与校验) ...
6
7      acquire(&semaphores[sem].lock);
8
9      // 释放资源
10     semaphores[sem].value += count;
11
12     // 唤醒等待的进程
13     wakeup(&semaphores[sem]);
14
15     release(&semaphores[sem].lock);
16
17     return 0;
18 }
```

`sem_signal()` (V 操作) 的实现流程如图??所示。

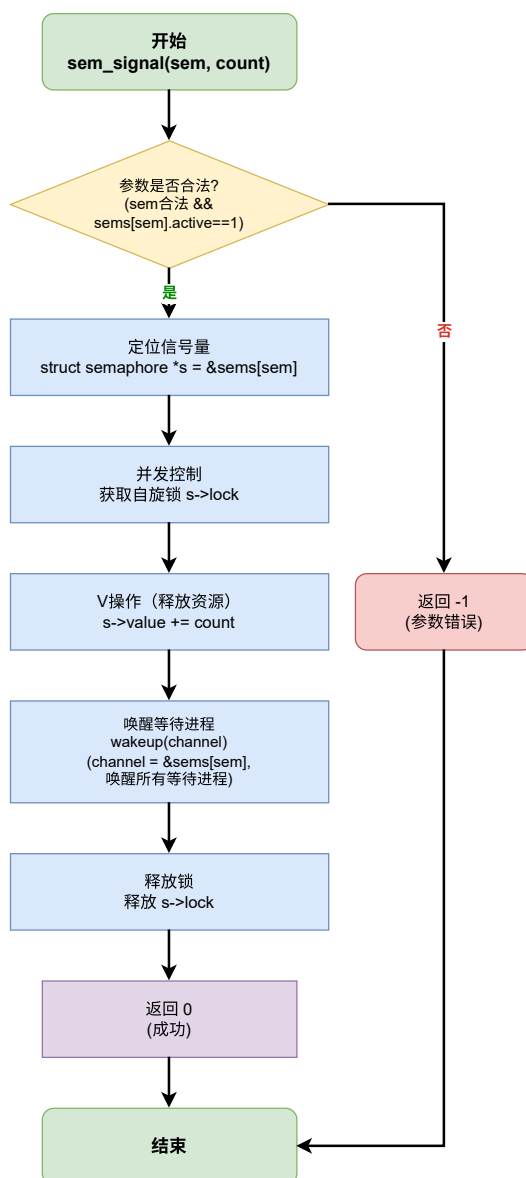


图 13 sem_signal (V 操作) 实现流程

4.4 任务三：使用信号量解决文件读写互斥

4.4.1 并发问题分析与临界区定义

任务要求 10 个进程并发地对同一个文件重复执行读-修改-写操作 50 次。这个操作不是原子性的。两个进程如果同时读到 10，分别加 1 后都写回 11，会导致结果错误。为了保证最终结果为 500，必须确保读-修改-写，这整个过程是互斥的，这个区域也就是临界区。

4.4.2 实现思路 (sem_test.c)

使用任务二实现的信号量来保护临界区。任务三的总体流程如图??所示。

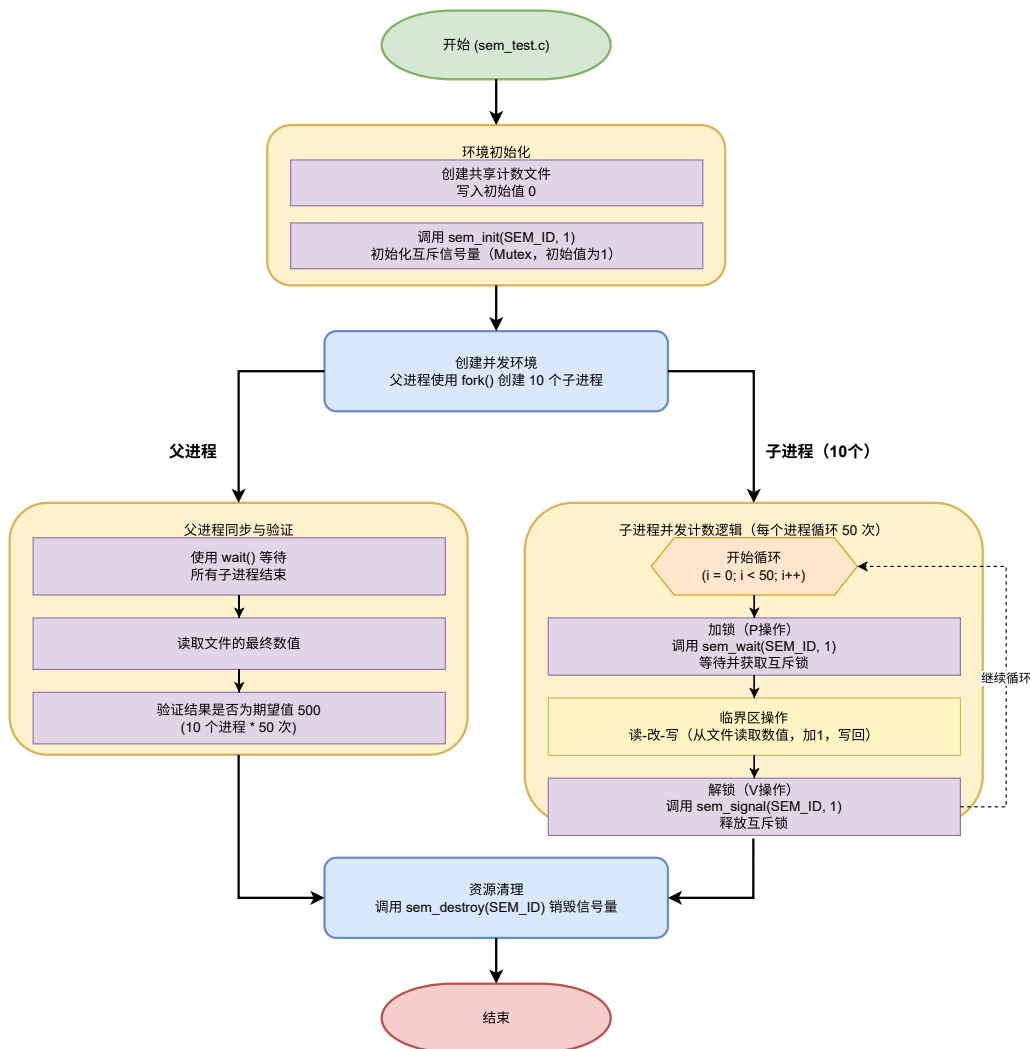


图 14 任务三总体实现流程

1. 初始化共享文件为 0。
2. 初始化互斥信号量: `sem_init(0, 1)` (初始值为 1, 作为互斥锁)。
3. 使用 `fork()` 创建 10 个子进程。
4. 子进程逻辑 (循环 50 次): P 操作进入临界区, 执行读写, V 操作退出临界区。
5. 父进程逻辑: `wait()` 所有子进程, 验证结果。

`sem_test.c` 的详细实现流程如图??所示。

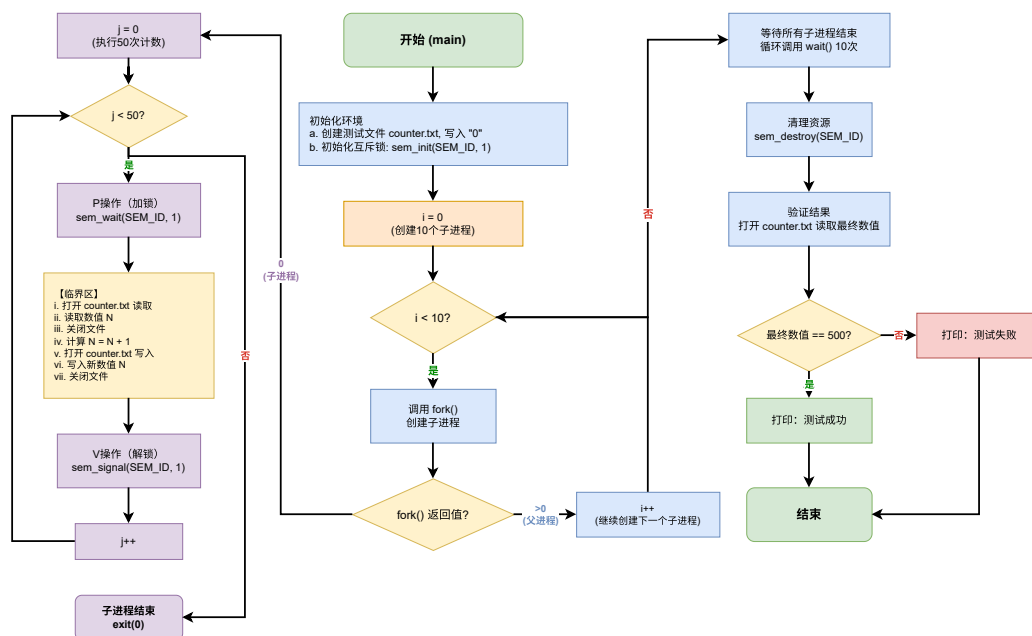


图 15 sem_test 程序详细实现流程

4.4.3 测试与验证

测试用例 1: 不使用信号量保护 (对照实验)

操作: 修改 sem_test.c, 注释掉 sem_wait() 和 sem_signal(). 重新编译并运行。

预期结果: Final value 有概率小于 500, Test FAILED。

```

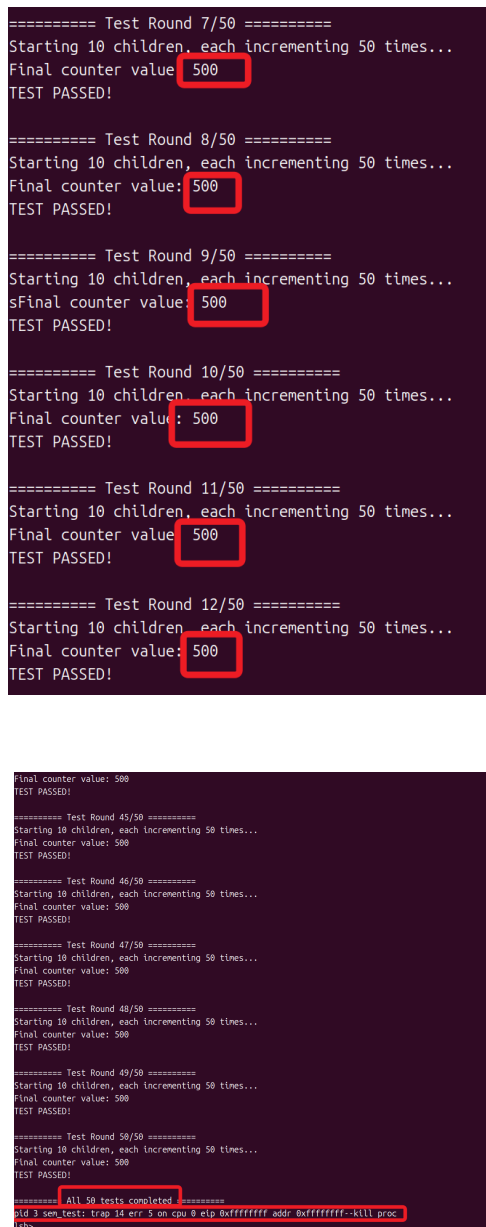
lsh> sem_test
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!
lsh> sem_test
Starting 10 children, each incrementing 50 times...
Final counter value: 307
TEST FAILED! Expected 500
lsh> sem_test
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!
lsh> sem_test
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!
lsh> sem_test
Starting 10 children, each incrementing 50 times...
Final counter value: 400
TEST FAILED! Expected 500
  
```

图 16 不使用信号量保护的运行结果, 有概率出现数据不一致

测试用例 2：使用信号量保护

操作：迭代运行 50 次包含信号量保护的 `sem_test.c`，查看输出是否有不符合预期。

预期结果：Final value: 500。Test PASSED。



```
===== Test Round 7/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 8/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 9/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 10/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 11/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 12/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

Final counter value: 500
TEST PASSED!

===== Test Round 45/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 46/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 47/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 48/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 49/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== Test Round 50/50 =====
Starting 10 children, each incrementing 50 times...
Final counter value: 500
TEST PASSED!

===== All 50 tests completed =====
$ cd 3 sem_test: trap 14 err: 5 on cpu 0 elp 0xffffffff addr 0xffffffff--kill proc:
$
```

图 17 五十次迭代测试判断信号量保护机制是否有效

通过对比实验，清晰地证明了信号量机制在解决并发互斥问题中的有效性。

5 实验总结

本次实验围绕进程优先级管理和信号量机制展开，通过实现三个具体的任务，我取得了以下技术成果和理论认识：

5.1 掌握了进程优先级的动态管理

我成功地扩展了 xv6 的进程控制块 (`struct proc`) 以包含优先级字段, 并实现了 `setpriority()` 系统调用和用户态 `nice` 命令。同时, 通过实现 `getptable()` 系统调用和 `ps` 命令, 我学会了如何将内核态的进程信息安全地 (通过数据复制) 暴露给用户态程序。

5.2 掌握了信号量机制的内核级实现

我成功地在内核中定义了信号量数据结构 (`struct semaphore`), 并实现了 `sem_init()`、`sem_destroy()`、`sem_wait()` (P 操作) 和 `sem_signal()` (V 操作) 四个核心系统调用。这标志着我掌握了这一经典同步机制的原理和实现细节。

5.3 解决了并发环境下的临界区保护

在任务三中, 我分析了多进程并发执行“读-修改-写”操作时产生的竞态条件。通过使用实现的信号量作为互斥锁 (Mutex), 我成功地保护了临界区, 确保了操作的原子性, 最终在 `sem_test.c` 中验证了 500 次累加操作的正确性。

5.4 深入理解了 Sleep 和 Wakeup 模型

信号量的实现核心在于进程的阻塞和唤醒。我熟练运用了 xv6 提供的 `sleep(chan, lock)` 和 `wakeup(chan)` 原语。我深刻理解了 `sleep` 如何原子性地释放锁并进入睡眠, 以及在被唤醒后如何重新获取锁。特别地, 我掌握了使用 `while` 循环进行条件检查 (而非 `if`), 以防止虚假唤醒 (Spurious Wakeup) 带来的问题。

6 实验心得

这次实验是继实现 Shell 之后的又一次深入内核的探索, 带给我的挑战和收获都非常大。

首先, 我深刻体会到了理论与实践的结合。在课堂上学习信号量时, P/V 操作似乎很清晰。但在实际实现时, 特别是处理原子性和阻塞唤醒的细节时, 才发现其复杂性。例如, `sem_wait` 中的 `while` 循环和 `sleep` 的配合是整个实现的关键。我意识到, 必须在持有锁的情况下检查条件, 并且依赖 `sleep` 来原子性地释放锁和阻塞。任何一步顺序的错误都可能导致死锁或同步失败。这个过程让我对锁的使用和进程状态转换有了更深刻的认识。

其次, 任务三的 `sem_test` 让我直观地看到了并发错误的发生以及同步机制的强大作用。当注释掉信号量保护后, 最终结果几乎总是小于 500, 这生动地展示了竞态条件

的危害。而加入信号量后，结果稳定为 500，这种从混乱到有序的过程让我对操作系统的核心职责——管理并发——有了更深的认同感。

最后，通过任务一实现 `ps` 和 `nice`，我熟练掌握了添加系统调用的完整流程，极大地提升了我在操作系统内核中进行开发和调试的能力。

7 附录：核心代码实现

7.1 附录 A：数据结构定义

7.1.1 A.1 ptable.h - 进程信息结构

```
ptable.h

1 // ptable.h - 进程信息结构定义，用于内核与用户态程序之间
  传递进程信息
2 #ifndef _PTABLE_H_
3 #define _PTABLE_H_
4
5 #include "param.h" // 引入NPROC常量定义
6
7 // proc_info结构体：轻量级进程快照，专为用户态程序(如ps
  命令)设计
8 struct proc_info {
9     int pid; // 进程ID，唯一标识一个进
  程
10    int ppid; // 父进程ID，如果没有父进
  程则为-1
11    int priority; // 进程调度优先级(1-20，
  数值越小优先级越高)
12    int mem_size; // 进程占用的内存大小(字
  节数)
13    enum procstate state; // 进程状态
14    char name[16]; // 进程命令名(最多16字节)
15 };
16
17 #endif
```

7.1.2 A.2 proc.h - 进程控制块扩展

```

proc.h (关键部分)

1 // 进程状态枚举
2 enum procstate {
3     UNUSED, // 进程槽位未使用
4     EMBRYO, // 进程正在创建中
5     SLEEPING, // 进程因等待某个事件而睡眠
6     RUNNABLE, // 进程就绪, 等待被调度执行
7     RUNNING, // 进程正在CPU上运行
8     ZOMBIE // 进程已终止, 等待父进程回收
9 };
10
11 // 进程控制块(PCB): 内核中描述每个进程的核心数据结构
12 struct proc {
13     uint sz; // 进程地址空间大小(字
14     节)
15     pde_t* pgdir; // 进程页目录的虚拟地址
16     char *kstack; // 进程内核栈的底部地址
17     enum procstate state; // 进程当前状态
18     int pid; // 进程ID
19     int priority; // 新增: 进程优先级
20     (1-20)
21     struct proc *parent; // 指向父进程的指针
22     struct trapframe *tf; // 保存用户态寄存器状态
23     struct context *context; // 进程上下文(用于上下
24     文切换)
25     void *chan; // 睡眠通道
26     int killed; // 非零表示进程被标记为
27     终止
28     struct file *ofile[NOFILE]; // 打开的文件描述符数组
29     struct inode *cwd; // 当前工作目录
30     char name[16]; // 进程名称
31     int sem_held[32]; // 新增: 记录进程持有的
32     信号量资源
33 };

```

7.1.3 A.3 proc.c - 信号量结构定义

```
proc.c (信号量定义)

1      // 内核信号量结构：用于进程间的同步与互斥
2      struct semaphore {
3          int value;                // 信号量的当前值(可用资源数
   量)
4          int active;              // 标志位：1表示该信号量槽位
   已初始化
5          struct spinlock lock;    // 保护value和active字段的自
   旋锁
6      };

7
8      // 全局信号量表：系统最多支持32个信号量
9      struct semaphore sema[32];
```

7.2 附录 B：进程管理系统调用实现

7.2.1 B.1 proc.c - setpriority 函数

```
setpriority 实现

1      // setpriority: 修改指定进程的优先级
2      // 参数: pid - 目标进程ID, priority - 新优先级值(1-20)
3      // 返回值: 0表示成功, -1表示失败
4      int
5      setpriority(int pid, int priority)
6      {
7          struct proc *p;          // 用于遍历进程表的指针
8          int found = 0;           // 标志: 是否找到目标进程
9
10         // 步骤1: 参数校验 - 优先级必须在[1, 20]范围内
11         if(priority < 1 || priority > 20)
12             return -1;
13
14         // 步骤2: 获取进程表锁, 确保互斥访问
15         acquire(&ptable.lock);
16
```

```
setpriority 实现

// 步骤3: 遍历进程表, 查找目标进程
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->pid == pid){
        p->priority = priority; // 修改优先级
        found = 1;
        break;
    }
}

// 步骤4: 释放进程表锁
release(&ptable.lock);

// 步骤5: 返回结果
return found ? 0 : -1;
}
```

7.2.2 B.2 proc.c - getptable 函数

```
getptable 实现

// getptable: 将内核进程表信息复制到用户空间
// 参数: ubuf - 用户空间缓冲区地址, size - 缓冲区大小
// 返回值: 0表示成功, -1表示失败
int
getptable(void *ubuf, int size)
{
    struct proc *p; // 进程表遍历指针
    struct proc_info pi; // 临时变量, 用于构造进
    程信息

    char *uptr = (char*)ubuf; // 用户缓冲区指针
    int i = 0; // 进程计数器

    // 步骤1: 缓冲区大小检查
    if(size < NPROC * sizeof(struct proc_info))
        return -1;
}
```

```
gettable 实现

// 步骤2: 获取进程表锁
acquire(&ptable.lock);

// 步骤3: 遍历进程表
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    // 清零临时结构体
    memset(&pi, 0, sizeof(pi));

    // 如果槽位正在使用, 填充信息
    if(p->state != UNUSED){
        pi.pid = p->pid;
        pi.ppid = (p->parent) ? p->parent->pid : -1;
        pi.priority = p->priority;
        pi.mem_size = p->sz;
        pi.state = p->state;
        safestrcpy(pi.name, p->name, sizeof(pi.name)
    );
    }

    // 步骤4: 复制到用户空间
    if(copyout(myproc()->pgdir,
        (uint)(uptr + i * sizeof(struct proc_info)),
        (char*)&pi, sizeof(pi)) < 0){
        release(&ptable.lock);
        return -1;
    }
    i++;
}

// 步骤5: 释放锁并返回
release(&ptable.lock);
return 0;
}
```

7.2.3 B.3 sysproc.c - 系统调用包装器

● ● ● 系统调用包装器

```
1      // sys_setpriority: setpriority系统调用包装器
2      int
3      sys_setpriority(void)
4      {
5          int pid, priority;
6
7          // 从系统调用参数栈中解析参数
8          if(argint(0, &pid) < 0)          // 参数0: 进程ID
9              return -1;
10         if(argint(1, &priority) < 0)      // 参数1: 优先级值
11             return -1;
12
13         // 调用内核实现函数
14         return setpriority(pid, priority);
15     }
16
17     // sys_gettable: gettable系统调用包装器
18     int
19     sys_gettable(void)
20     {
21         char *buf;    // 用户空间缓冲区指针
22         int size;     // 缓冲区大小
23
24         // 解析参数并验证用户地址有效性
25         if(argptr(0, &buf, sizeof(struct proc_info) * NPROC)
26         ||
27         argint(1, &size))
28             return -1;
29
30         // 调用内核实现函数
31         return gettable(buf, size);
32     }
```

7.3 附录 C：用户态程序实现

7.3.1 C.1 ps.c - 进程查看命令

```
ps.c

1      #include "types.h"
2      #include "stat.h"
3      #include "user.h"
4      #include "param.h"    // 包含NPROC常量定义
5      #include "mmu.h"
6      #include "proc.h"     // 包含procstate枚举定义
7      #include "ptable.h"   // 包含proc_info结构定义
8
9      // 进程状态字符串数组：将状态枚举转换为可读字符串
10     static char *states[] = {
11         [UNUSED]    "UNUSED ", // 补空格对齐到8字符
12         [EMBRYO]    "EMBRYO ",
13         [SLEEPING]  "SLEEPING",
14         [RUNNABLE]  "RUNNABLE",
15         [RUNNING]   "RUNNING ",
16         [ZOMBIE]    "ZOMBIE "
17     };
18
19     int
20     main(int argc, char *argv[])
21     {
22         struct proc_info pinfo[NPROC]; // 接收进程信息的缓
23         冲区
24
25         // 步骤1：清零缓冲区，防止打印垃圾数据
26         memset(pinfo, 0, sizeof(pinfo));
27
28         // 步骤2：调用getptable系统调用获取进程表信息
29         if(getptable(pinfo, sizeof(pinfo)) < 0){
30             printf(2, "ps: getptable failed\n");
31             exit();
32         }
```

```
ps.c

// 步骤3: 打印表头
printf(1, "PID      PPID      PRI      MEM      STATE
CMD\n");

// 步骤4: 遍历并打印每个进程的信息
for(int i = 0; i < NPROC; i++){
    // 跳过未使用的进程槽位
    if(pinfo[i].pid == 0)
        continue;

    // 打印PID
    printf(1, "%d      ", pinfo[i].pid);

    // 打印PPID (-1则显示N/A)
    if(pinfo[i].ppid == -1)
        printf(1, "N/A      ");
    else
        printf(1, "%d      ", pinfo[i].ppid);

    // 打印优先级和内存大小
    printf(1, "%d      %d      ", pinfo[i].priority,
pinfo[i].mem_size);

    // 打印进程状态
    if(pinfo[i].state >= 0 && pinfo[i].state < 6)
        printf(1, "%s      ", states[pinfo[i].state]);
    else
        printf(1, "???      ");

    // 打印进程名称
    printf(1, "%s\n", pinfo[i].name);
}

exit();
}
```

7.3.2 C.2 nice.c - 优先级修改命令

```
nice.c

1      #include "types.h"
2      #include "stat.h"
3      #include "user.h"
4      #include "fcntl.h"
5
6      int
7      main(int argc, char *argv[])
8      {
9          int pid, priority;
10
11         // 步骤1: 参数检查 - 需要两个参数: pid和priority
12         if(argc != 3){
13             printf(2, "Usage: nice pid priority\n");
14             exit();
15         }
16
17         // 步骤2: 解析命令行参数
18         pid = atoi(argv[1]);          // 第一个参数: 进程ID
19         priority = atoi(argv[2]);     // 第二个参数: 优先级值
20
21         // 步骤3: 参数校验 - 优先级必须在[1, 20]范围内
22         if (priority < 1 || priority > 20) {
23             printf(2, "nice: priority must be between 1 and
24 20\n");
25             exit();
26         }
27
28         // 步骤4: 调用setpriority系统调用
29         if(setpriority(pid, priority) < 0){
30             printf(2, "nice: failed to set priority (pid %d
31 not found?)\n", pid);
32         } else {
33             printf(1, "nice: priority of process %d set to %
34 d\n", pid, priority);
35         }
36     }
```

```
nice.c

33
34     exit();
35 }
```

7.4 附录 D：信号量实现

7.4.1 D.1 proc.c - sem_init 函数

```
sem_init 实现

1 // sem_init: 初始化一个信号量
2 // 参数: sem - 信号量ID(0-31), value - 信号量初始值
3 // 返回值: 0表示成功, -1表示失败
4 int
5 sem_init(int sem, int value)
6 {
7     // 步骤1: 参数校验
8     if(sem < 0 || sem >= 32)
9         return -1;
10
11     // 步骤2: 获取该信号量的锁
12     acquire(&sema[sem].lock);
13
14     // 步骤3: 检查是否已被初始化
15     if(sema[sem].active == 1) {
16         release(&sema[sem].lock);
17         return -1; // 已在使用, 不允许重复初始化
18     }
19
20     // 步骤4: 初始化信号量
21     sema[sem].value = value; // 设置初始值
22     sema[sem].active = 1;    // 标记为活跃状态
23
24     // 步骤5: 释放锁
25     release(&sema[sem].lock);
26
27     return 0;
```

```
sem_init 实现

28     }
```

7.4.2 D.2 proc.c - sem_destroy 函数

```
sem_destroy 实现

1      // sem_destroy: 销毁一个信号量
2      // 参数: sem - 信号量ID(0-31)
3      // 返回值: 0表示成功, -1表示失败
4      int
5      sem_destroy(int sem)
6      {
7          // 步骤1: 参数校验
8          if(sem < 0 || sem >= 32)
9              return -1;
10
11         // 步骤2: 获取锁
12         acquire(&sema[sem].lock);
13
14         // 步骤3: 检查是否处于活跃状态
15         if(sema[sem].active == 0) {
16             release(&sema[sem].lock);
17             return -1;
18         }
19
20         // 步骤4: 销毁信号量
21         sema[sem].active = 0; // 标记为未使用
22         sema[sem].value = 0; // 清零计数值
23
24         // 步骤5: 释放锁
25         release(&sema[sem].lock);
26
27         return 0;
28     }
```

7.4.3 D.3 proc.c - sem_wait 函数 (P 操作)

```
sem_wait 实现

1 // sem_wait: P操作, 申请资源, 资源不足则阻塞
2 // 参数: sem - 信号量ID, count - 需要申请的资源数量
3 // 返回值: 0表示成功, -1表示失败
4 int
5 sem_wait(int sem, int count)
6 {
7     // 步骤1: 参数校验
8     if(sem < 0 || sem >= 32)
9         return -1;
10
11     // 步骤2: 获取信号量锁
12     acquire(&sema[sem].lock);
13
14     // 步骤3: 检查是否处于活跃状态
15     if(sema[sem].active == 0){
16         release(&sema[sem].lock);
17         return -1;
18     }
19
20     // 步骤4: 循环检查资源是否足够 (核心逻辑)
21     // 使用while而非if, 防止虚假唤醒
22     while(sema[sem].value < count){
23         // 资源不足, 进程进入睡眠
24         // sleep的参数1: 睡眠通道 (使用信号量地址)
25         // sleep的参数2: 当前持有的锁
26         // sleep会原子性地释放锁并进入SLEEPING状态
27         sleep(&sema[sem], &sema[sem].lock);
28
29         // 被唤醒后, 回到while开头重新检查条件
30
31         // 防御性检查: 信号量是否在睡眠期间被销毁
32         if(sema[sem].active == 0){
33             release(&sema[sem].lock);
34             return -1;
35         }
36     }
```

```
sem_wait 实现

36     }
37
38     // 步骤5: 资源充足, 消耗资源
39     sema[sem].value -= count;
40
41     // 步骤6: 记账, 用于进程退出时的自动资源回收
42     myproc()->sem_held[sem] += count;
43
44     // 步骤7: 释放锁
45     release(&sema[sem].lock);
46
47     return 0;
48 }
```

7.4.4 D.4 proc.c - sem_signal 函数 (V 操作)

```
sem_signal 实现

1 // sem_signal: V操作, 释放资源并唤醒等待者
2 // 参数: sem - 信号量ID, count - 需要释放的资源数量
3 // 返回值: 0表示成功, -1表示失败
4 int
5 sem_signal(int sem, int count)
6 {
7     // 步骤1: 参数校验
8     if(sem < 0 || sem >= 32)
9         return -1;
10
11     // 步骤2: 获取信号量锁
12     acquire(&sema[sem].lock);
13
14     // 步骤3: 检查活跃状态
15     if(sema[sem].active == 0){
16         release(&sema[sem].lock);
17         return -1;
18     }
```

```
sem_signal 实现

19
20 // 步骤4: 释放资源, 增加信号量的值
21 sema[sem].value += count;
22
23 // 步骤5: 销账, 减少当前进程的资源持有计数
24 if(myproc()->sem_held[sem] >= count) {
25     myproc()->sem_held[sem] -= count;
26 } else {
27     // 防御性处理: 持有计数不足, 清零
28     myproc()->sem_held[sem] = 0;
29 }
30
31 // 步骤6: 唤醒所有等待该信号量的进程
32 // wakeup将所有在该通道上睡眠的进程状态改为RUNNABLE
33 wakeup(&sema[sem]);
34
35 // 步骤7: 释放锁
36 release(&sema[sem].lock);
37
38 return 0;
39 }
```

7.4.5 D.5 sysproc.c - 信号量系统调用包装器

```
信号量系统调用包装器

1 // sys_sem_init: sem_init系统调用包装器
2 int
3 sys_sem_init(void)
4 {
5     int sem, value;
6
7     if (argint(0, &sem) < 0) // 解析参数0: 信号量ID
8         return -1;
9     if (argint(1, &value) < 0) // 解析参数1: 初始值
10        return -1;
```

● ● ● 信号量系统调用包装器

```
11
12         return sem_init(&sem, 0, value);
13     }
14
15     // sys_sem_destroy: sem_destroy系统调用包装器
16     int
17     sys_sem_destroy(void)
18     {
19         int sem;
20
21         if (argint(0, &sem) < 0)
22             return -1;
23
24         return sem_destroy(&sem);
25     }
26
27     // sys_sem_wait: sem_wait系统调用包装器
28     int
29     sys_sem_wait(void)
30     {
31         int sem, count;
32
33         if (argint(0, &sem) < 0)           // 解析参数0: 信号量ID
34             return -1;
35         if (argint(1, &count) < 0)        // 解析参数1: 资源数量
36             return -1;
37
38         return sem_wait(&sem, count);
39     }
40
41     // sys_sem_signal: sem_signal系统调用包装器
42     int
43     sys_sem_signal(void)
44     {
45         int sem, count;
46     }
```

```
信号量系统调用包装器

47     if (argint(0, &sem) < 0)
48     return -1;
49     if (argint(1, &count) < 0)
50     return -1;
51
52     return sem_signal(sem, count);
53 }
```

7.5 附录 E：信号量测试程序

7.5.1 E.1 sem_test.c - 完整实现

```
sem_test.c

1     #include "types.h"
2     #include "stat.h"
3     #include "user.h"
4     #include "fcntl.h"
5
6     // 测试参数定义
7     #define NUM_CHILDREN 10           // 并发子进程数量
8     #define TARGET_COUNT_PER_CHILD 50 // 每个子进程的循环
   次数
9     #define COUNTER_FILE "counter"   // 共享计数器文件名
10    #define SEM_ID 0                 // 使用0号信号量
11
12    // test_counter: 信号量互斥测试函数
13    void
14    test_counter()
15    {
16        int i, j;
17        int fd;
18        int val;
19
20        // ===== 第一部分：初始化 =====
21
22        // 步骤1：创建共享计数器文件并写入初始值0
```

```
sem_test.c

23     fd = open(COUNTER_FILE, O_CREATE | O_RDWR);
24     if(fd < 0){
25         printf(1, "Error: create counter file failed\n")
26     ;
27         exit();
28     }
29     val = 0;
30     write(fd, &val, sizeof(int)); // 写入4字节整数
31     close(fd);
32
33     // 步骤2: 初始化互斥信号量 (初始值为1, 作为互斥锁)
34     if(sem_init(&SEM_ID, 1) < 0){
35         printf(1, "Error: sem_init failed\n");
36         exit();
37     }
38
39     printf(1, "Starting %d children, each incrementing %d times...\n",
40           NUM_CHILDREN, TARGET_COUNT_PER_CHILD);
41
42     // ===== 第二部分: 创建并发子进程 =====
43
44     for(i = 0; i < NUM_CHILDREN; i++){
45         int pid = fork();
46         if(pid < 0){
47             printf(1, "fork failed\n");
48             exit();
49         }
50
51         if(pid == 0){
52             // --- 子进程代码 ---
53             for(j = 0; j < TARGET_COUNT_PER_CHILD; j++){
54
55                 // P操作: 申请进入临界区
56                 if(sem_wait(&SEM_ID, 1) < 0){
57                     printf(1, "Child: sem_wait failed\n")
```

```
sem_test.c

);

57         exit();
58     }
59
60     // ===== 临界区开始 =====
61
62     // 读取当前计数值
63     fd = open(COUNTER_FILE, O_RDONLY);
64     if(fd < 0){
65         printf(1, "Child: open read failed\n
66     ");
67         exit();
68     }
69     if(read(fd, &val, sizeof(int)) != sizeof
70     (int)){
71         printf(1, "Child: read failed\n");
72         exit();
73     }
74     close(fd);
75
76     // 更新计数值 (加1)
77     val++;
78
79     // 写回新值
80     fd = open(COUNTER_FILE, O_WRONLY);
81     if(fd < 0){
82         printf(1, "Child: open write failed\
83     n");
84         exit();
85     }
86     if(write(fd, &val, sizeof(int)) !=
87     sizeof(int)){
88         printf(1, "Child: write failed\n");
89         exit();
90     }
91     close(fd);
```

```
sem_test.c

88
89 // ===== 临界区结束 =====
90
91 // V操作：离开临界区
92 if(sem_signal(SEM_ID, 1) < 0){
93     printf(1, "Child: sem_signal failed\
n");
94     exit();
95 }
96 }
97
98 exit(); // 子进程完成后退出
99 }
100 }
101
102 // ===== 第三部分：等待子进程并验证结果 =====
103
104 // 父进程等待所有子进程退出
105 for(i = 0; i < NUM_CHILDREN; i++){
106     wait();
107 }
108
109 // 读取最终计数值
110 fd = open(COUNTER_FILE, O_RDONLY);
111 read(fd, &val, sizeof(int));
112 close(fd);
113
114 printf(1, "Final counter value: %d\n", val);
115
116 // 验证结果（期望值：10 * 50 = 500）
117 if(val == NUM_CHILDREN * TARGET_COUNT_PER_CHILD){
118     printf(1, "TEST PASSED!\n");
119 } else {
120     printf(1, "TEST FAILED! Expected %d\n",
NUM_CHILDREN * TARGET_COUNT_PER_CHILD);
121 }
122 }
```

```
sem_test.c

123
124 // ===== 第四部分：清理资源 =====
125
126 sem_destroy(SEM_ID); // 销毁信号量
127 unlink(COUNTER_FILE); // 删除测试文件
128
129 return;
130 }
131
132 int
133 main(int argc, char *argv[])
134 {
135     test_counter();
136     return 0;
137 }
```